DICE FEST 2026 · Babak final 30 Oktober 2026 · Tim BISMILLAH

Q&A Juri 03: Lima Pertanyaan Sulit tentang Penggabungan, Koreksi, dan Kebocoran

Status. Rujukan rinci untuk P1 sampai P5. Status bagian RISIKO sudah diperbarui di 06 Tanya jawab v2.

Daftar isi ada di slide berikutnya: ketuk judul bagian untuk melompat ke sana.

Daftar isi

Pengantar	3
P1. Pergeseran bobot penggabungan	5
P2. Bobot adaptif `A2xM2xp2`	8
P3. Koreksi galat wilayah	12
P4. Dugaan kebocoran pada kNN *out-of-fold*	16
P5. Peran `pipeline.assemble`	19
Lima pertanyaan lanjutan dan jawaban singkat	23

Pengantar

Disusun dari sudut pandang dewan juri data science yang kritis, berdasarkan kode yang benar-benar dijalankan:

- `blend.py`;
- `postprocess.py`;
- `pipeline.assemble` dan `pipeline._assemble`;
- `train_lib.run` dan `train_lib.Data._knn_features`;
- `fb07b.py`;
- `configs/resid_knn.json` (ditulis oleh sel 22).

Nomor sel dihitung dari 0, dengan nomor eksekusi Jupyter di dalam kurung.

Cara memakai. Setiap pedoman jawaban disusun dengan urutan: inti jawaban, apa yang dilakukan kode, asal angkanya, dan pengakuan alat bantu. Bagian bertanda **RISIKO** adalah keputusan atau angka yang tidak punya justifikasi lengkap di notebook. Pelajari bagian itu lebih dulu.

Pengantar (lanjutan)

Pernyataan umum soal alat bantu (dipakai bila juri bertanya langsung):

> "Kami memakai Claude sebagai alat bantu pemrograman: menulis dan merapikan kode, menjalankan eksperimen, dan menyusun dokumentasi. Semua nilai prediksi dihasilkan oleh kode open-source yang dijalankan di komputer kami, bukan oleh model bahasa. Setiap komponen hanya dipertahankan bila memperbaiki skor validasi, dan alasan desainnya bisa kami jelaskan."

P1. Pergeseran bobot penggabungan

Pertanyaan juri. "Bobot gabungan Anda di-fit pada prediksi *out-of-fold* seluruh data latih, lalu dipakai pada prediksi data uji. Prediksi data uji adalah rata-rata 10 model fold, sedangkan prediksi *out-of-fold* berasal dari satu model per fold. Bagaimana Anda tahu bobotnya tidak *overfit* dan tidak bergeser ketika dipakai di data uji?"

Apa yang dilakukan kode.

- `blend.nnls_blend` memakai NNLS (kuadrat terkecil dengan bobot tidak negatif, `scipy.optimize.nnls`). Inputnya matriks prediksi *out-of-fold*: 429.303 baris × 28 anggota gas/EUI, atau × 13 anggota listrik. Untuk kolom gas, setiap baris diberi skala luas lantai (`row_scale=A`), karena galat gas sama dengan luas dikali galat EUI.
- Setiap prediksi *out-of-fold* berasal dari model yang tidak melihat baris tersebut (`train_lib.run`).
- `pipeline._assemble` langkah 1 mem-fit bobot pada seluruh data latih. Skor jujurnya dihitung oleh `blend.cv_blend_score`: bobot di-fit pada 4/5 baris, lalu diuji pada 1/5 sisanya (KFold 5, seed 0).

P1. Pergeseran bobot penggabungan (lanjutan)

Angka dan asalnya.

- **Skor jujur gabungan** (sel 35 = In [27], diringkas di sel 37): gas 0,24494, EUI 0,65023, listrik 0,31587. Angka ini dihitung sebelum koreksi wilayah dan penyesuaian akhir.
- **Bobot global** (`summary.json` yang ditulis run notebook):
  - jumlah bobot tidak nol: EUI 21 dari 28 anggota, gas 23 dari 28, listrik 13 dari 13;
  - bobot terbesar hanya 0,11 sampai 0,14, jadi tidak ada satu model yang mendominasi;
  - jumlah bobot 1,004 sampai 1,008, karena NNLS tidak memaksa jumlahnya tepat 1.
- **Perkembangan LB publik** (sel 37 = In [28]): setiap langkah dipilih dari *holdout set*, dan LB turun dari 0,25025 (v12) ke 0,24913 (final) sesuai perkiraan itu.

Pedoman jawaban. "Bobot tidak pernah dinilai pada baris yang dipakai untuk mem-fit-nya. Skor jujur di notebook memakai bobot dari empat per lima data lalu diuji di seperlima sisanya. Jumlah parameternya kecil, 13 sampai 28 bobot untuk 429 ribu

P1. Pergeseran bobot penggabungan (lanjutan)

baris, dan semuanya tidak boleh negatif, sehingga ruang untuk *overfit* sempit. Bukti di data uji: setiap kali kami mengubah penggabungan, leaderboard bergerak sesuai perkiraan *holdout*."

RISIKO: tim perlu mempelajari ini. Perbedaan antara prediksi *out-of-fold* (satu model per fold) dan prediksi uji (rata-rata 10 model) tidak diuji secara terpisah. Jawaban jujur: "Kami belum mengukur pergeseran ini secara khusus. Rata-rata 10 model biasanya sedikit lebih halus daripada satu model, dan NNLS tanpa batasan jumlah 1 bisa menyerap sebagian perbedaan skala. Bukti tidak langsung kami adalah kecocokan perkiraan *holdout* dengan leaderboard di setiap langkah."

Pengakuan alat bantu. "Fungsi penggabungan dan validasinya ditulis dengan bantuan Claude. Bagian yang kami periksa sendiri adalah memastikan skor jujur memakai bobot yang tidak melihat baris uji."

P2. Bobot adaptif `A2xM2xp2`

Pertanyaan juri. "`A2xM2xp2` terdengar seperti *tuning* berlebihan. Berapa parameter yang Anda tambahkan, apa artinya secara intuitif, dan mengapa dua titik simpul, bukan tiga?"

Apa yang dilakukan kode.

- `blend.parse_vc("A2xM2xp2")` menghasilkan tiga variabel pengondisi, masing-masing dengan 2 titik simpul (*knot*):
  - A = luas lantai (`bld_dim_06`);
  - M = jumlah meter (`bld_dim_12`);
  - p = prediksi gabungan global.
- `blend.size_basis` mengubah setiap variabel menjadi peringkatnya di data latih (0 sampai 1), lalu membuat fungsi segitiga. Dengan 2 simpul, fungsinya hanyalah garis lurus 1 − t dan t, yaitu bobot "ujung rendah" dan "ujung tinggi".

P2. Bobot adaptif `A2xM2xp2` (lanjutan)

- `blend.vc_basis` mengalikan ketiganya menjadi 2 × 2 × 2 = 8 kolom. Setiap anggota punya 8 bobot "pojok" (A rendah/tinggi × M rendah/tinggi × p rendah/tinggi). Bobot sebuah kawasan adalah interpolasi di antara pojok-pojok itu.
- `blend.nnls_blend_vc` mem-fit semua bobot pojok dengan NNLS, sehingga semuanya tidak negatif.
- Variabel pengondisi hanya berupa fitur (A, M) dan prediksi (p), tidak ada target. Peringkat data uji dihitung terhadap distribusi data latih.
- Jumlah parameter: 28 × 8 = 224 bobot untuk gas/EUI dan 13 × 8 = 104 untuk listrik, di-fit pada 429.303 baris.

Angka dan asalnya.

- **LB publik** (sel 37): v19 0,24934 menjadi v19vc "bobot gabungan yang menyesuaikan" 0,24925, selisih −0,00009.

P2. Bobot adaptif `A2xM2xp2` (lanjutan)

- **Validasi bersarang 5-fold saat pengembangan** (catatan keputusan D012 di `reports/DECISIONS.md`, di luar notebook):
  - A2 saja: −0,000039;
  - A × M: −0,000077;
  - level prediksi: −0,000067;
  - A2xM2xp2: −0,000124;
  - bentuk yang lebih kaya (A3xM2xp2, A2xM2xp3) tidak lebih baik.

Pedoman jawaban. "Gagasannya sederhana: model yang paling akurat di kawasan kecil belum tentu paling akurat di kawasan besar. Daripada satu set bobot untuk semua kawasan, setiap model punya bobot di 'ujung kecil' dan 'ujung besar', dan kawasan di antaranya mendapat campuran. Bentuk ini kami pilih dengan validasi bersarang. Tiga simpul per variabel tidak lebih baik daripada dua, jadi kami memakai yang paling sederhana. Di leaderboard, perubahan ini menurunkan skor 0,00009."

P2. Bobot adaptif `A2xM2xp2` (lanjutan)

RISIKO: tim perlu mempelajari ini.

- Skor jujur gabungan adaptif **tidak dicetak di notebook**. Sel 35 hanya mencetak skor jujur gabungan global.
- Bukti angkanya ada di catatan pengembangan D012 dan di tabel LB.
- Jawaban jujur: "Pemilihan bentuk ini berasal dari perbandingan validasi bersarang saat pengembangan. Notebook hanya menerapkan bentuk terpilih. Kami belum menguji variabel pengondisi lain di luar luas, meter, dan level prediksi."
- Pelajari juga cara kerja `size_basis`. Juri bisa meminta contoh: kawasan dengan peringkat luas 0,8 mendapat 20% bobot "ujung kecil" dan 80% bobot "ujung besar" untuk variabel A.

Pengakuan alat bantu. "Implementasi basis segitiga ditulis dengan bantuan Claude. Ide dasarnya muncul dari analisis galat yang menunjukkan model berbeda unggul di ukuran kawasan berbeda, dan keputusannya diambil dari validasi."

P3. Koreksi galat wilayah

Pertanyaan juri. "Anda menggeser prediksi data uji dengan rata-rata galat tetangga. Galat itu dihitung dari target data latih. Bagaimana bobot koreksi λ ditentukan, mengapa ada λ negatif, dan apakah ini bukan kebocoran?"

Apa yang dilakukan kode (`pipeline._assemble` langkah 2, `postprocess.all_features`):

1. **Galat setiap baris latih:** r = target − prediksi gabungan. Untuk gas, targetnya luas dikali EUI aktual.
2. **Fitur koreksi untuk setiap kawasan uji:**
   - Rata-rata galat kawasan latih di zona yang sama, petak iklim yang sama (`env_climate_01/03/05`), dan jendela ±30 nomor subzona (`WIN = 30`). Rata-rata ini dikecilkan dengan faktor n/(n + 30) (`K = 30`) supaya kelompok kecil tidak berpengaruh besar.

P3. Koreksi galat wilayah (lanjutan)

- Rata-rata galat k kawasan latih termirip di beberapa ruang kemiripan ("full", "era", "small", "fc") dalam jendela ±W nomor subzona. Rata-rata ini dikecilkan dengan faktor n/(n + 5) (`KSHR = 5`).
- Jumlah konfigurasi tetangga: gas 6, EUI 13, listrik 5 (`resid_knn.json`, sel 22).
- Skala per target: gas sebagai pergeseran per luas yang dikali luas; EUI sebagai rata-rata berbobot luas; listrik sebagai pergeseran relatif yang dikali prediksi.

3. **Koreksi = F × λ.** λ dibekukan di `configs/resid_knn.json`: 9 koefisien untuk gas, 16 untuk EUI, 8 untuk listrik. Koreksi hanya diterapkan pada baris uji.

Asal λ. Saat pengembangan, semua model dilatih ulang tanpa *holdout set* 30%. Galat dari 70% data itu dipakai sebagai tetangga, lalu λ dipilih dengan regresi linear supaya galat pada 30% sisanya paling kecil. Proses fit ini tidak ada di notebook; notebook hanya menerapkan nilai beku, seperti dijelaskan di teks bagian 3 (sel 30).

Angka dan asalnya.

- **Besar koreksi** (sel 35): rms/σ gas 0,02047, EUI 0,04344, listrik 0,01195.

P3. Koreksi galat wilayah (lanjutan)

- **LB publik** (sel 37): v12 0,25025 menjadi v15 "koreksi galat wilayah" 0,24954, selisih −0,0007. Ini perbaikan terbesar dalam tabel tersebut.
- **λ gas** (sel 22):
  - kelompok: zona 0,453, petak iklim 0,346, jendela subzona −0,020;
  - enam konfigurasi tetangga: 0,037; 0,126; 0,079; 0,072; 0,124; 0,156.

Mengapa bukan kebocoran. Yang dipakai hanya target data latih, yang memang diberikan panitia. Target data uji tidak pernah dipakai. λ dipilih di *holdout set* yang tidak ikut menyusun tetangga.

Pedoman jawaban. "Setelah digabung, galat model masih berkelompok: kalau tetangga yang mirip sama-sama ditebak terlalu rendah, kawasan ini kemungkinan juga terlalu rendah. Kami memakai pola itu dengan hati-hati. Rata-ratanya dikecilkan bila tetangganya sedikit, bobotnya dipilih di data yang disisihkan, dan koreksinya kecil, sekitar 1 sampai 4 persen dari σ. Hasilnya adalah perbaikan terbesar kami di leaderboard."

P3. Koreksi galat wilayah (lanjutan)

RISIKO: tim perlu mempelajari ini.

- **λ negatif** (gas −0,020; EUI −0,069 dan −0,045; listrik −0,009). Jawaban jujur: "λ adalah koefisien regresi linear pada fitur yang saling berkorelasi. Rata-rata tetangga dari berbagai ruang saling tumpang tindih, sehingga satu koefisien kecil bisa bertanda negatif tanpa makna fisik tersendiri. Yang kami validasi adalah efek totalnya. Versi dengan penalti ridge pernah diuji di audit akhir dan tidak lebih baik secara konsisten."
- **Konstanta K = 30, WIN = 30, KSHR = 5** tidak dijelaskan di notebook. Jawaban jujur: "Nilai ini ditetapkan saat pengembangan dan diperiksa ulang di audit akhir. Mengubahnya tidak memperbaiki *holdout*, tetapi kami belum memetakan sensitivitasnya secara lengkap."

Pengakuan alat bantu. "Kode pencarian tetangga yang cepat ditulis dengan bantuan Claude. Logika koreksinya, yaitu rata-rata galat tetangga yang dikecilkan dan bobot yang di-fit di data disisihkan, kami pahami dan bisa kami jelaskan dengan contoh."

P4. Dugaan kebocoran pada kNN *out-of-fold*

Pertanyaan juri. "Fitur kawasan termirip memakai target tetangga. Bagaimana Anda menjamin baris validasi tidak memakai targetnya sendiri atau target baris lain di fold yang sama? Apakah skor *out-of-fold* Anda masih jujur?"

Apa yang dilakukan kode (`train_lib.run`, pemanggilan `knn_features` per fold):

- **Baris latih fold k (bagian a):** `knn_features(a, "oof")` membagi bagian a menjadi 5 bagian dalam (KFold, seed 42). Fitur setiap baris dihitung dari 4 bagian dalam lainnya, jadi baris tidak pernah menjadi tetangga dirinya sendiri.
- **Baris validasi fold k (bagian b):** `knn_features(a, ("train", b))` mengambil tetangga hanya dari bagian a. Target bagian b tidak dipakai sama sekali.
- **Data uji:** `knn_features(a, "test")` mengambil tetangga dari bagian a. Prediksi akhir adalah rata-rata 10 model fold.
- **Fitur lain dengan pola yang sama:**
  - Rata-rata target wilayah (`te_features`) memakai pola cross-fit yang sama. Opsi `te_full` (leave-one-out) tidak dipakai oleh satu pun dari 41 anggota gabungan (`members.yaml`).

P4. Dugaan kebocoran pada kNN *out-of-fold* (lanjutan)

- Fitur tambahan di `fb07b.py`, dipakai 13 anggota (`knn_x`), juga memakai fungsi cross-fit yang sama.

Angka dan asalnya.

- **LB publik** (sel 37): v15 0,24954 menjadi v16 "fitur kawasan paling mirip di dalam model" 0,24943.
- **Galat EUI model tunggal** turun 0,0022 (teks bagian 2, sel 28).
- **Skema 10-fold acak sah** karena data uji diambil acak per baris (Lampiran A, sel 54 dan 55): kawasan uji di subzona baru 8,14% (harapan 8,17%), AUC validasi adversarial 0,4997.

Pedoman jawaban. "Setiap fitur yang memakai target dibuat dengan cross-fit dua lapis. Baris validasi hanya melihat tetangga dari sembilan fold lain, dan baris latih hanya melihat tetangga dari empat bagian dalam yang lain. Tidak ada baris yang melihat targetnya sendiri. Ada satu perbedaan yang kami sadari: fitur baris latih dihitung dari 80% bagian latih, sedangkan fitur baris validasi dan uji dari 100%. Ini membuat fitur latih sedikit lebih berisik, jadi arahnya pesimistis, bukan bocor."

P4. Dugaan kebocoran pada kNN *out-of-fold* (lanjutan)

RISIKO: tim perlu mempelajari ini.

- **Uji kebocoran otomatis tidak ada di notebook.** Uji label acak dijalankan saat pengembangan. Jawaban jujur: "Notebook tidak memuat uji itu. Bukti di notebook adalah struktur cross-fit di atas, pemeriksaan pemisahan data di Lampiran A, dan kecocokan skor validasi dengan leaderboard."
- **Sumber optimisme kecil yang sudah diakui di bagian 6 (sel 36):** XGBoost dan LightGBM berhenti lebih awal berdasarkan fold validasinya sendiri. Sebutkan ini sebelum juri menemukannya.

Pengakuan alat bantu. "Fungsi kNN dengan jendela nomor subzona dan pemrosesan per blok ditulis dengan bantuan Claude. Aturan cross-fit-nya, yaitu siapa boleh menjadi tetangga siapa, adalah bagian yang kami periksa baris demi baris karena di situlah kebocoran biasanya terjadi."

P5. Peran `pipeline.assemble`

Pertanyaan juri. "`pipeline.assemble` tampak seperti kotak hitam. Apa saja yang terjadi di dalamnya, mana yang di-fit dan mana yang dibekukan, dan bagaimana Anda menjamin hasilnya bisa direproduksi?"

Apa yang dilakukan kode (`pipeline.assemble` lalu `pipeline._assemble`): 0. Jumlah thread BLAS dibatasi menjadi 1 (`threadpool_limits(1)`), karena jumlah thread bisa mengubah digit terakhir perkalian matriks.

1. **Penggabungan:**
   - Memuat prediksi *out-of-fold* dan prediksi uji 41 anggota.
   - Gabungan NNLS global beserta skor jujurnya (dicetak), lalu gabungan adaptif `A2xM2xp2`.
   - Porsi bangunan bersih = round(100 × `bld_dim_02` / `bld_dim_01`, 2).
   - Adopsi diambil dari `adopt_v2` lalu dipotong ke 0 sampai 100.
2. **Koreksi galat wilayah** dengan λ dan konfigurasi tetangga yang dibekukan (P3).
3. **Penyesuaian akhir** (2b di kode), dengan parameter `postproc` yang dibekukan:
   - **S1:** prediksi kawasan uji dengan 2 sampai 3 meter dikali (1 + δ): gas +20,1%, EUI +18,4%, listrik +18,7%. Untuk 4 meter: +6,4%, +3,2%, dan +7,1%.

P5. Peran `pipeline.assemble` (lanjutan)

- **S3** (gas dan EUI): koreksi distribusi normal terpotong di batas gas maksimum data latih. Prediksi digeser turun sebesar $k \times s \times \varphi(b)/\Phi(b)$, dengan $s = p \times \sqrt{(a0 + b0/M)}$ dan $b = (\text{batas} - p)/s$. Nilainya: gas $k = 0{,}505$, $a0 = 0{,}0081$, $b0 = 0{,}393$; EUI $k = 0{,}657$, $a0 = 0{,}0084$, $b0 = 0{,}387$.
- **S4:** semua prediksi dipotong ke batas yang tersirat dari data latih: gas tidak melebihi maksimum latih, EUI tidak di bawah minimum latih, dan listrik per luas berada di antara minimum dan maksimum latih.

4. **Salinan duplikat:** 77 kawasan uji yang identik dengan kawasan latih (`dup_fix`) disalin targetnya.
5. **Pemeriksaan dan keluaran:** urutan kolom dan `id` harus sama dengan `sample_submission.csv`, dan semua nilai harus berhingga. Setelah itu ditulis CSV, sidik sha256, dan `summary.json`.

Mana yang di-fit dan mana yang dibekukan.

- **Di-fit di dalam `assemble`:** bobot NNLS global dan adaptif, pada prediksi *out-of-fold*.
- **Dibekukan di `resid_knn.json`** (berasal dari *holdout set* dan audit): bentuk `A2xM2xp2`, konfigurasi tetangga, λ, serta parameter S1 dan S3.

P5. Peran `pipeline.assemble` (lanjutan)

- **Dihitung langsung dari data latih:** batas S4 dan batas gas maksimum.

Angka dan asalnya.

- **sha256** `aec745ab2114e6e1...d66e` (sel 35).
- **Penyesuaian akhir:** LB publik v19nca 0,24920 menjadi final 0,24913 (sel 37).
- **77 duplikat** tercatat di `summary.json`.

Pedoman jawaban. "Assemble adalah satu fungsi yang menjalankan urutan tetap: gabung, koreksi, sesuaikan batas, salin duplikat, periksa, lalu tulis. Yang di-fit di dalamnya hanya bobot gabungan. Semua besaran lain dipilih di data yang disisihkan lalu dibekukan di satu berkas konfigurasi, sehingga notebook tidak memilih apa pun berdasarkan leaderboard. Hasilnya byte demi byte sama setiap kali dijalankan, dan sidik sha256-nya tercetak di notebook."

RISIKO: tim perlu mempelajari ini.

- **Asal nilai S1** (misalnya +20,1%) **dan S3** (k, a0, b0). Nilainya di-fit di skrip audit pengembangan (catatan D015), bukan di notebook. Yang perlu dipahami tim:
  - S1 adalah rata-rata kekurangan prediksi semua model pada kelompok kawasan bermeter sangat sedikit, diukur di prediksi *out-of-fold*.

P5. Peran `pipeline.assemble` (lanjutan)

  - S3 memakai rumus rata-rata distribusi normal terpotong (*inverse Mills ratio*), dengan skala galat dari model a + b/M. Faktor k di bawah 1 berarti koreksi teoretisnya hanya diterapkan sebagian, sesuai hasil *holdout*.
- **Jawaban jujur:** "Parameter ini di-fit pada data latih dan *holdout*, lalu dibekukan; notebook hanya menerapkannya. Efeknya kecil, 0,00007 di leaderboard, dan sesuai perkiraan *holdout*. Kami belum menguji bentuk koreksi lain di luar normal terpotong."

Pengakuan alat bantu. "Struktur `assemble` dan pemeriksaan reprodusibilitasnya ditulis bersama Claude. Alasan setiap langkah bisa kami jelaskan, termasuk mengapa thread BLAS dibatasi: tanpa itu, digit terakhir hasil bisa berbeda antarmesin."

Lima pertanyaan lanjutan dan jawaban singkat

1. **"Mengapa validasi acak, bukan validasi spasial per zona?"** Data uji diambil acak per baris. Notebook membuktikannya di Lampiran A: kawasan uji di subzona baru 8,14% (harapan pengacakan 8,17%), dan validasi adversarial tidak bisa membedakan data latih dari data uji (AUC 0,4997). Validasi per zona justru menguji situasi yang tidak terjadi di data uji.
2. **"Bukankah menyalin adopsi per subzona dan 77 baris duplikat itu kebocoran?"** Tidak. Yang disalin adalah target data latih, yang memang boleh dipakai. Adopsi konstan di 100% subzona data latih (sel 25), jadi 91,86% kawasan uji yang subzonanya dikenal mendapat nilai yang sama. Target data uji tidak pernah dipakai.
3. **"Seberapa besar peran AI dalam solusi ini?"** Claude dipakai sebagai alat bantu menulis kode, menjalankan eksperimen, dan menyusun dokumen. Prediksi dihasilkan oleh model open-source yang dilatih di komputer kami. Keputusan dipilih berdasarkan validasi, dan alasan setiap komponen bisa kami jelaskan, seperti pada lima jawaban di atas.

Lima pertanyaan lanjutan dan jawaban singkat (lanjutan)

4. **"Mengapa SHAP dari model penjelas, bukan dari gabungan 41 model?"** Gabungan memakai rata-rata target wilayah yang bukan tuas kebijakan, dan 41 model sulit dibaca. Model penjelas hanya memakai karakteristik yang terukur. Kecocokannya dilaporkan (R^2 0,563 untuk EUI, 0,747 untuk gas per meter, 0,491 untuk listrik per meter; sel 41), jadi batas keyakinannya terlihat.
5. **"Apakah hasil Anda bisa direproduksi orang lain?"** Ya. Seed dikunci, mode deterministik dipakai, dan thread BLAS dibatasi. Notebook mencetak sidik sha256 berkas submisi (sel 35), yang sama dengan berkas yang dikirim. Pelatihan penuh butuh sekitar 20 jam di GPU 16 GB. Tanpa pelatihan, notebook butuh sekitar 5 jam (catatan di sel 0).